

LUT core-module ablation: three generations of lookup math

Comparison table

2026-09-14

Protocol

Architecture: LUT transformer, $E = 384$, 6 layers, 6 attention heads. FFN = **CompressionMultiHeadLUT** with $H = 4$ heads $\times$ $tph = 128$ tables ($P = H \cdot tph = 512$ tables per token), $d_{in} = d_{out} = 48$, $n = 8$ anchor pairs per table ($K = 2^n = 256$ cells); **compress** $384 \rightarrow 192$, **decompress** $192 \rightarrow 384$. The vanilla baseline (Table B row V) replaces this FFN with the standard dense one, $384 \rightarrow 1536 \rightarrow 384$, GELU, no bias. **Training:** 16,000 steps, batch size 48 (24576 tokens), untied embeddings, LUT tables without weight decay. **Eval:** validation bits per byte (bpb), **evaluate_bpb_fixed**, 48 sequences $\times$ 100 steps, first 12 sequences skipped.

Counting convention

- Core LUT only, per token, one layer. The **compress/decompress** projections are identical in every row and excluded (147,456 MACs at eval).
- Row V has no LUT and no projections, so its figures are the whole dense FFN ($E \rightarrow 4E \rightarrow E$, $E = 384$); compare them with a LUT row plus the projections. Its 1,536 GELUs are not decomposed into FLOPs.
- **FLOP** = one scalar float operation ($+$, $-$, $\times$, $\div$, a comparison, $|\cdot|$, $\exp$, $\log \sigma$, σ). **MAC** = a multiply accumulated into a sum; 1 MAC = 2 FLOPs, included in the FLOP column. Memory reads and integer work are not counted.
- **Eval** = exactly what inference computes, per token. **Train** = the whole cost of one training step: the computation training shares with inference, all gradients, including per-table gradients of per-layer scalars, and anything computed only in training — a score or blend weight the eval read does not use, and a surrogate’s score, blend weights and gradients (not the surrogate’s value, note b). A hard-forward row’s eval is therefore the hard read alone, while its train adds the surrogate. Train = eval + the training-only work, except for row 3.2 int8 (note d).

Table A — forward function and gradient function

Per variant, (a) is the function computed on the forward pass and (b) the function whose gradient training uses: the weights always receive the gradient of (a), while the input, the temperatures and the learned per-layer scalars receive the gradient of (b) with the table weights held constant.

Variant	(a) Forward: the function computed	(b) Backward: the function whose gradient is used
GEN 1 — Spiking Manifesto basic model		
1.1 Hard forward, two-alternatives backward	$y = \sum_t W_t[c_t]$, summed over tables t : each of table t 's n anchor pairs gives a signed margin $u_j = z[a_j] - z[b_j]$; the n signs $[u_j > 0]$ are the bits of the binary index $c_t \in \{0, \dots, 2^n - 1\}$ of the row read from $W_t \in \mathbb{R}^{K \times d}$, $K = 2^n$.	The 2-cell blend $\sum_t [(1 - U_t)W_t[c_t] + U_tW_t[c'_t]]$, $U_t = U(u_{j^*})$, $U(u) = 0.5/(1 + u)$, with the least-confident pair $j^* = \arg \min_j u_j $ and c' (c with bit j^* flipped) held fixed. The pairing with (a) is exact as a stop-gradient composite; the blend is C^1 across a bit flip and jumps only where j^* switches.
1.2 Soft forward, two-alternatives backward	(a) = (b): the blend of 1.1(b), applied at eval too.	
GEN 2 — FastMHL math		
2.1 Hard forward, all-alternatives backward (all K cells)	$y = \sum_t W_t[c_t]$, as 1.1(a).	With $b_j(\ell) = +1$ if the bit of ℓ for pair j is 1, else -1 , and the squashed margin $\hat{m}_j = u_j/(T_s + u_j) \in (-1, 1)$: $s_\ell = \sum_j b_j(\ell) \hat{m}_j$ scores how well row ℓ 's bit pattern agrees with the actual signs of the margins (small margins count little either way). All K rows of each table are read, softmax-weighted within that table: $\sum_t \sum_{\ell < K} \text{softmax}_\ell(s_\ell/T_{sel}) W_t[\ell]$. T_s, T_{sel} : per-layer learned scalars. Globally C^1 in the input; never uses the address, whose row is the one with the largest weight.
2.2 Hybrid_smooth two-alternatives forward, all-alternatives backward	Reads two cells, the addressed c_t and the alternative c'_t (c_t with the bit of the least-confident pair $j^* = \arg \min_j u_j $ flipped), blended by a weight set by that pair's margin: $y = \sum_t [(1 - w_t)W_t[c_t] + w_tW_t[c'_t]]$, $w_t = \sigma(-2\rho_{j^*}/T_{sel}) \in (0, \frac{1}{2}]$, $\rho_j = u_j /(T_s + u_j)$, σ the logistic sigmoid; applied at eval too.	The same softmax-weighted read of all K rows as 2.1(b), with the same T_s, T_{sel} parameters as (a).
2.3 Hard forward, 2-alternatives backward	The hard read of 1.1(a).	The softmax of 2.1(b) restricted to the two cells c_t and c'_t (c_t with the bit of the least-confident pair $j^* = \arg \min_j u_j $ flipped) and renormalised over that pair. The weight on c'_t is $\sigma(-2\rho_{j^*}/T_{sel})$, $\rho_j = u_j /(T_s + u_j)$, σ the logistic sigmoid: exactly 2.2(a)'s w_t , so this is 2.2(a)'s 2-cell blend.
2.4 Hybrid_smooth two-alternatives forward, 2-alternatives backward	(a) = (b): the 2-cell blend of 2.2(a). The backward keeps only 2 of the K rows, the addressed c_t and its least-confident neighbour c'_t , which are exactly the two cells the forward reads, so the restricted softmax equals the forward weight w_t and forward and backward coincide; unlike 2.2, the input gradient reaches only pair j^* .	
GEN 3 — LightMHL math (LookupFFN-inspired): learned margin score		
3.1 The Gen-3 form: learned margin score, one cell, per-layer (β, γ)	(a) = (b): $y = \sum_t s_t W_t[c_t]$ with the address c_t held fixed; $s_t = S_t P_t^\gamma$, $S_t = \sum_j m_j$, $P_t = \prod_j \sigma(\beta m_j)$, $m_j = u_j $, computed as $S_t \exp(\gamma \sum_j \log \sigma(\beta m_j))$; β, γ learned per layer from (2, 1), where s_t is LookupFFN's margin score. Discontinuous at cell boundaries, since $\sigma(0) > 0$.	
3.2 The Gen-3 form, two alternatives, per-layer (β, γ, τ)	(a) = (b): each table reads two cells, $y = \sum_t s_t [(1 - v_t)W_t[c_t] + v_tW_t[c'_t]]$, with s_t as in 3.1 over all n margins and c'_t the cell c_t with the bit of the smallest margin $j^* = \arg \min_j m_j$ flipped (c_t, c'_t, j^* held fixed). v_t is a softmax over the two cells with logits 0 and $-2m_{j^*}/\tau$ (flipping bit j lowers $\langle u, b \rangle$ by $2m_j$), i.e. $v_t = \sigma(-2m_{j^*}/\tau) \in (0, \frac{1}{2}]$; τ learned per layer from 0.5. The input gradient reaches all n margins through s_t and m_{j^*} alone through v_t . Continuous across a bit flip ($v_t = \frac{1}{2}$ there); jumps where j^* switches.	

continued on next page

Variant	(a) Forward: the function computed	(b) Backward: the function whose gradient is used
3.3 / 3.4 Hard forward, Gen-3 backward (3.1 / 3.2)	The hard read of 1.1(a), $y = \sum_t W_t[c_t]$; no score at eval.	3.1(b) (for 3.3) or 3.2(b) (for 3.4) on detached weights, with each score divided by the detached mean score of its head ($\hat{s}_t = s_t / \text{sg}(\bar{s} + \epsilon)$), which keeps these gradients bounded (without it training diverged, b5b0f2c3); the weights get (a)'s gradient. Kept as evidence that a hard forward is weak, not as candidates.

Table B — cost and results

Per token, one layer, core LUT only (row V: the whole dense FFN); $P = 512$, $H = 4$, $n = 8$, $K = 256$, $d = 48$, $E = 384$. **Validation bpb**: final_val_bpb from the committed `summary.json` of the cell's run folder `abl_NN` in `experiments/ffn_replacement/lut_ablation`, one seed each; in parentheses the difference from V (1.165148). Two vanilla seeds at 16K differ by 0.00335 (exp_n_0135 1.165147, exp_n_0176 1.161798), so smaller differences are inside single-seed noise.

Variant	Eval FLOPs	Train FLOPs	Eval MACs	Train MACs	Validation bpb	Validation bpb, +TV ($\lambda = 10$)	Implementation (class used)
V	$2 \cdot 2E \cdot 4E =$ 2,359,296 + 1,536 GELUs	$2 \cdot 2E \cdot 4E + 2 \cdot 4E \cdot 4E + 4E =$ 7,079,424 + 1,536 GELUs, 1,536 GELU'	$2E \cdot 4E =$ 1,179,648	$2E \cdot 4E + 4E \cdot 4E =$ 3,538,944	1.165148 , abl_10; parent exp_n_0135 1.165147; seed 2 1.161798 (exp_n_0176)	n/a: no LUT tables	Dense FFN (ffn_type='dense'): Linear(384, 1536, bias=False), GELU, Linear(1536, 384, bias=False).
1.1	$P(2n + d) =$ 32,768	$P(6d + 4n + 9) =$ 168,448	0	$P \cdot 2d =$ 49,152	1.199672 (+0.0345), abl_11	1.193616 (+0.0285), abl_12	CompressionMultiHeadLUT(lut_impl='gen1'): MultiHeadLut(smooth_mode=False, n_alternatives=1, uncertainty_mode=INVERSE_L1, weights_init='uniform'), each head's anchors in its own slice.
1.2	$P(4d + 4n + 4) =$ 116,736	$P(12d + 4n + 13) =$ 317,952	$P \cdot 2d =$ 49,152	$P \cdot 6d =$ 147,456	1.185924 (+0.0208), abl_13	1.181644 (+0.0165), abl_14	As 1.1 with smooth_mode=True.
2.1	$P(2n + d) =$ 32,768	$P(2Kd + 4nK + 9K + 13n + 2d) =$ 18,059,264^a	0	$P(Kd + 2nK) =$ 8,388,608	1.181035 (+0.0159), abl_07; parent exp_n_0121 1.180622	1.174661 (+0.0095), abl_15	FastMultiHeadLut(forward_mode='hard', backward_topk=0).
2.2	$P(2kd + 4n + 5) =$ 117,248 , $k = 2$ cells read	$P(2Kd + 4nK + 9K + 15n + 4kd + 5) =$ 18,217,472^a	$Pkd =$ 49,152	$P(Kd + 2nK + 2kd) =$ 8,486,912	1.169401 (+0.0043), abl_01	1.162635 (−0.0025), abl_16	FastMultiHeadLut(forward_mode='hybrid_smooth', backward_topk=0).
2.3	$P(2n + d) =$ 32,768	$P(2kd + 4nk + 9k + 14n + 2d - 1) =$ 246,272 , $k = 2$ cells	0	$P(kd + 2nk) =$ 65,536	1.203803 (+0.0387), abl_02	1.196908 (+0.0318), abl_17	FastMultiHeadLut(forward_mode='hard', backward_topk=1); backward_topk counts the cells kept besides c_t .
2.4	$P(2kd + 4n + 5) =$ 117,248	$P(6kd + 13n + 20) =$ 358,400	$Pkd =$ 49,152	$P3kd =$ 147,456	1.179331 (+0.0142), abl_03	1.174482 (+0.0093), abl_18	FastMultiHeadLut(forward_mode='hybrid_smooth', backward_topk=1).
3.1	$P(2kd + 7n + 1) =$ 78,336 , $k = 1b$	$P(6kd + 17n + 9) =$ 221,696^b	$Pkd =$ 24,576	$P3kd =$ 73,728	1.169578 (+0.0044), abl_08; parent exp_g_0248 1.169328	1.163849 (−0.0013), abl_09; parent exp_g_0249 1.163698	LightMultiHeadLUT(confidence_form='learned_margin', learned_margin_init=(0, 2, 1), learned_margin_freeze_g=True), with g the log-gain, held at 0.

continued on next page

Variant	Eval FLOPs	Train FLOPs	Eval MACs	Train MACs	Validation bpb	Validation bpb, +TV ($\lambda = 10$)	Implementation (class used)
3.2	$P(2kd + 8n + 8) = 135,168$, $k = 2^b$	$P(6kd + 18n + 31) = 384,512^b$	$Pkd = 49,152$	$P3kd = 147,456$	1.159969 (−0.0052), abl_04	1.154739 (−0.0104), abl_05	As 3.1 with read_top_n=2, read_tau=0.5, read_tau_learnable=True.
3.2 int8	$P(7n + 16) + Hd = 37,056^d$	$P(6kd + 17n + 57) = 393,728$, $k = 2^d$	0	$P3kd = 147,456$	not run	1.158890 (−0.0063), abl_48; unquantised rerun abl_46 1.155175	As 3.2 with quant_mode='p2_int8': power-of-two cell weights, int8 cells (section at the end).
3.3	$P(2n + d) = 32,768$	$P(4d + 17n + 11) + H = 173,572^b$	0	$Pd = 24,576$	1.210356 (+0.0452), abl_19	1.198918 (+0.0338), abl_20	As 3.1, hard forward. ^c
3.4	$P(2n + d) = 32,768$	$P(6d + 18n + 32) + H = 237,572^b$	0	$P2d = 49,152$	1.201621 (+0.0365), abl_21	1.196248 (+0.0311), abl_22	As 3.2, hard forward. ^c

Notes to Table B.

- ^a Dominated by $2Kd$: every table's K rows are read; several [tokens, 512, 256] fp32 buffers of about 12.9 GiB at 24,576 tokens.
- ^b Per table, eval: n comparisons, n differences, n absolute values, n products βm_j , $n \log \sigma$, the two sums, $\times \gamma$, exp and the final product $(7n + 1)$, plus the scaled rows $(2kd)$; 3.2 adds the argmin $(n - 1)$, the logit $-2m_{j^*}/\tau$ (2), the two-way softmax (4) and s_t times the two weights (2). Train adds, as training-only work: the row terms $(4kd)$, the input term $(8n + 3)$ and the per-table accumulation of the parameter gradients $(\partial \log \gamma$ 3, $\partial \log \beta$ $2n + 2$); 3.2 adds ∂s_t from two rows (3), ∂v_t (2), the softmax gradient (5), ∂m_{j^*} (3) and $\partial \log \tau$ (2). The parameter gradients are scalar multiplies, so they add no MACs. 3.3, 3.4: inference performs the hard read alone, while training also reads a surrogate on detached weights (3.1's / 3.2's read with the score rescale). In the count, eval is the hard read; train adds the surrogate's score (and for 3.4 the argmin and two-way softmax) and the score rescale $(+2P + H / +3P + H)$ FLOPs, no MACs), with each row's score-scaled weight update replaced by one plain update at c_t : training-only work $P(3d + 15n + 9) + 2P + H = 140,804$ (3.3) and $P(5d + 16n + 29) + 3P + H = 204,804$ (3.4). Not counted: the value of the surrogate f and the $+f - \text{sg}(f)$ pair, which cancel exactly (the implementation evaluates them). **Training-only work** (train − eval), FLOPs / MACs, so the gradient cost can be read in isolation: V 4,720,128 + 1,536 GELU' / 2,359,296; 1.1 135,680 / 49,152; 1.2 201,216 / 98,304; 2.1 18,026,496 / 8,388,608; 2.2 18,100,224 / 8,437,760; 2.3 213,504 / 65,536; 2.4 241,152 / 98,304; 3.1 143,360 / 49,152; 3.2 249,344 / 98,304; 3.2 int8 356,864 / 147,456 (not train − eval, note d); 3.3 140,804 / 24,576; 3.4 204,804 / 49,152.
- ^c 3.3, 3.4 run on code with the score rescale (b5b0f2c3); without it all four diverged (final bpb 2.7–3.2).
- ^d 3.2 int8, counted under the same convention as note b (eval = what inference computes, train = the whole training step), with five choices stated. **Eval, per table** (the deployed integer read): n comparisons and n differences (the address), n absolute values, the sum S $(n - 1)$, n products βm_j , $n \log \sigma$ and their sum $(n - 1)$, the argmin $(n - 1) - 7n - 3$, shared with 3.2; then q_t : m_{j^*} times the per-layer constant $2/(\tau \ln 2)$, $+\frac{1}{2}$, floor, clamp to $[0, J]$ (5); c_{q_t} : the test $q_t < C$, 2^{-q_t} , $+1$, $\log_2$ (4); k'_t : $\log_2 S_t$, $\times \gamma$, $\div \ln 2$, $+$, $-c_{q_t}$, $+\frac{1}{2}$, floor (7); the window and drop tests: $k'_t < -3$, $k'_t > 4$, $q_t > 3$ (3). Total $7n + 16$ per table. The shifts $k'_t + 6$, $k'_t + 6 - q_t$ and the two-cell shift-add over the int8 cells are integer work, so they count 0 FLOPs and 0 MACs; the one conversion of the Hd int32 accumulators to float per token counts Hd . **Training-only work, per table**: the float read of the two cells at their straight-through weights $(2kd, kd)$ MACs; the score and blend values eval does not form — exp and the product $S_t \cdot \exp$ (2), the logit (2), the two-way softmax (4), s_t times the two weights (2); the straight-through ratios: $2^{k'_t}$ and $2^{k'_t - q_t}$ as floats (2), the clamp of the two exact weights (2) and the two divisions (2); their use in the gradient, the two multiplies ∂w -ratio (2); and 3.2's gradients unchanged, $4kd + 10n + 23$ with $2kd$ MACs. Total $6kd + 10n + 41$ with $3kd$ MACs, i.e. 356,864 FLOPs and 147,456 MACs. **Train** = the per-table integers $(7n + 16)$, as in eval) + this training-only work = $P(6kd + 17n + 57) = 393,728$ FLOPs and 147,456 MACs. **This row is the one exception to train = eval + training-only work** ($37,056 + 356,864 = 393,920$): the $Hd = 192$ int32→float conversions per token are eval-only — training computes the same per-table integers but then performs the float straight-through read instead of the integer read, so it never converts ($393,728 = 37,056 - 192 + 356,864$). **Choices**: (1) operations outside the FLOP list (floor, $\log_2$, 2^{-q}) count 1 FLOP each, as exp and $\log \sigma$ do in note b; c_{q_t} is counted as computed, as the code does — as a lookup in a C -entry table it would cost 0; as in note b, the log-gain g (held at 0) is not counted, nor are straight-through value pairs of the form $f - \text{sg}(f)$, which cancel exactly. (2) The int32→float conversion counts 1 FLOP per accumulator element (Hd per token), in eval only. (3) Worst case: every table read and both cells taken; skipping ($k'_t < -3$) and dropping the second cell ($q_t > 3$) make the measured average lower, so these figures are upper bounds (the integer read has no FLOPs to save, but training's float read does). (4) The float straight-through read exists only in training and is counted in train, not in eval. (5) The per-step table quantisation (per weight: $|\cdot|$, the running max, $\div 2^{e_{h,i}}$, $+\frac{1}{2}$, floor, two clamp comparisons, $\times 2^{e_{h,i}} = 8$ FLOPs over $PKd = 6,291,456$ weights, plus 4 per exponent over Hd) is training only and is amortised over the 24,576 tokens of a step: +2,048 FLOPs and no MACs per token per layer. As with the TV penalty, it is stated here and not included in the columns.

Hard forward. With the Gen-3 backward unchanged, replacing the blended forward by the hard read costs +0.041 (3.3 vs 3.1), +0.035 (3.3 vs 3.1, +TV), +0.042 (3.4 vs 3.2) and +0.042 bpb (3.4 vs 3.2, +TV), the same direction as Gens 1–2 (1.1, 2.1, 2.3 against 1.2, 2.2, 2.4): a blending or confidence-weighted forward is necessary. Rows 3.3 and 3.4 are kept for this comparison only.

Cell TV regularisation (Table B’s +TV column)

- **Penalty:** $\lambda \cdot \frac{1}{6} \sum_{\text{layers}} \frac{1}{P n 2^{n-1}} \sum_t \sum_{(c,c')} \|W_t[c] - W_t[c']\|^2$, squared L2 over the cell pairs (c, c') that differ in one bit, $\lambda = 10$; its gradient is added once per step, before gradient clipping. Training only: +6,144 FLOPs and +2,048 MACs per token per layer (amortised over 24,576 tokens); inference unchanged.
- **All three generations:** the trainer’s penalty (`lut_tv_penalty`) covers `MultiHeadLut`, `FastMultiHeadLut` and `LightMultiHeadLUT` tables, whose Hamming-1 cell pairs were checked through each module’s read path, and `build_model` refuses $\lambda > 0$ on a model whose tables it cannot reach.

Quantised variant of 3.2 +TV: power-of-two cell weights and int8 cells

The best LUT cell, 3.2 +TV, is quantised so that the deployed read of a table uses only cell lookups, additions and bit shifts over 8-bit integer cells; the few per-table scalars that steer it stay in floating point. The notation is Table A’s: table t has margins $u_j = z[a_j] - z[b_j]$, $m_j = |u_j|$, addressed cell c_t , alternative cell c'_t (the bit of $j^* = \arg \min_j m_j$ flipped), score $s_t = S_t P_t^\gamma$ and blend weight $v_t = \sigma(-2m_{j^*}/\tau)$ on c'_t ; $T = 128$ tables per head, cell width $d = 48$. Rounding is $\text{round}(\xi) = \lfloor \xi + \frac{1}{2} \rfloor$ (half up); a multiply by 2^m is a shift by m bits. It is implemented as `quant_mode='p2_int8'` of `LightMultiHeadLUT`.

From the blend to shifts and adds

3.2’s read of table t is $s_t[(1 - v_t)W_t[c_t] + v_tW_t[c'_t]]$. Three steps turn its two weights into powers of two.

- **Factorise (exact).** With $r_t = v_t/(1 - v_t) = e^{-2m_{j^*}/\tau}$: $s_t[(1 - v_t)W_t[c_t] + v_tW_t[c'_t]] = s_t(1 - v_t)(W_t[c_t] + r_t W_t[c'_t])$.
- **Round r_t .** $q_t = \text{clamp}(\lfloor 2m_{j^*}/(\tau \ln 2) + \frac{1}{2} \rfloor, 0, J)$, so $r_t \approx 2^{-q_t}$, and $c_q = \log_2(1 + 2^{-q})$ for $q < C$, 0 otherwise, so $\log_2(1 + r_t) \approx c_{q_t}$; $J = 64$, $C = 8$. If $q_t > Q = 3$ the second cell’s weight is below 1/8 and it is dropped.
- **Round $s_t(1 - v_t)$.** Since $\log_2(s_t(1 - v_t)) = \log_2 s_t - \log_2(1 + r_t)$, set $k'_t = \text{round}(\log_2 s_t - c_{q_t})$, with the score in the log domain, $\log_2 s_t = \log_2 S_t + \frac{\gamma}{\ln 2} \sum_j \log \sigma(\beta m_j)$, so P_t is never formed.
- **Exponent window.** $k'_t \in [-3, 4]$ ($L = 8$ values): a table with $k'_t < -3$ is skipped, one with $k'_t > 4$ is clamped to 4.

Table t then reads $2^{k'_t}(W_t[c_t] + 2^{-q_t}W_t[c'_t])$, or $2^{k'_t}W_t[c_t]$ when the second cell is dropped.

Training and integer cells

- **Straight-through cell weights.** Training reads $\hat{w}_t W_t[c_t] + \hat{w}'_t W_t[c'_t]$ with $\hat{w}_t = s_t(1 - v_t) \text{sg}(2^{k'_t}/(s_t(1 - v_t)))$ and $\hat{w}'_t = s_t v_t \text{sg}(2^{k'_t - q_t}/(s_t v_t))$: in value exactly $2^{k'_t}$ and $2^{k'_t - q_t}$ (0 when dropped or skipped), with the gradients of $s_t(1 - v_t)$ and $s_t v_t$ times the same ratios, so β, γ, τ and the input keep learning and skipped tables can come back. The trained read is the deployed read.
- **Int8 cells.** Each weight is an 8-bit integer times a power of two shared by head h and output channel i : $e_{h,i} = \lceil \log_2 \max_{t \in h, c} |W_t[c]_i| \rceil - 7$, $\hat{W} = \text{clamp}(\text{round}(W/2^{e_{h,i}}), -128, 127)$; training uses $2^{e_{h,i}} \hat{W}$ in value with the gradient passed to the float W unchanged, and $e_{h,i}$ is recomputed from W on every forward pass.
- **Scales cost nothing at runtime.** Nothing sits between the table sum and `decompress`, and $e_{h,i}$ is common to head h ’s tables, so after training $2^{e_{h,i}}$ is multiplied into column $dh + i$ of `decompress`’s weight matrix M (once, at export).

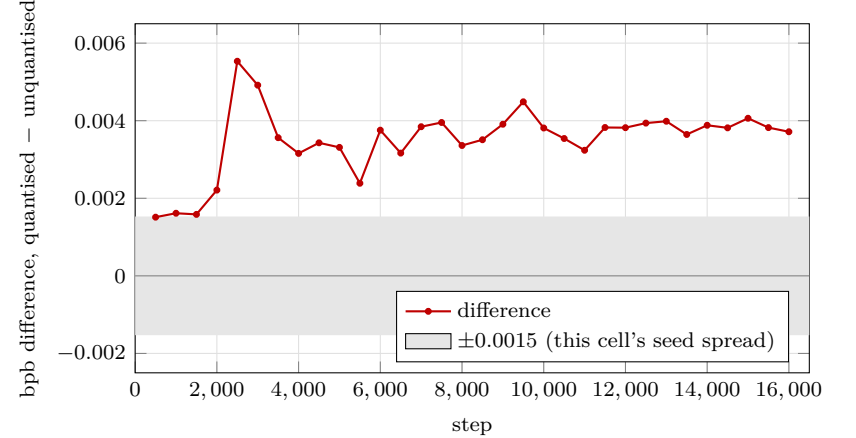
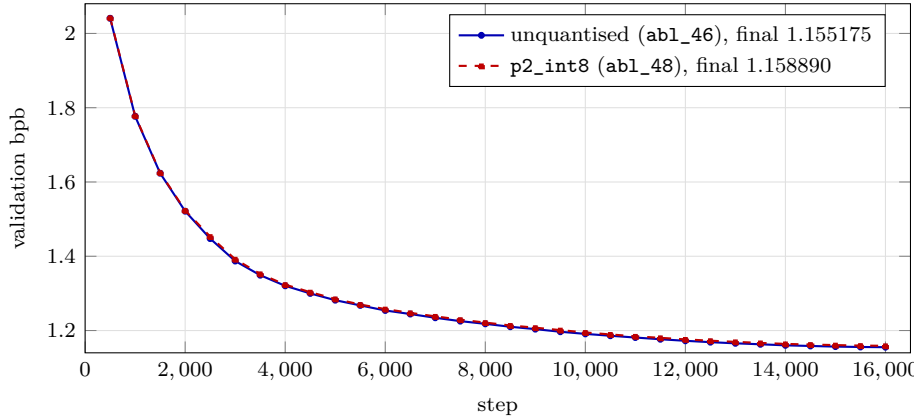
The deployed head sum, in integers kept in units of 2^{-6} so that every shift is a left shift, is

$$\hat{y}_h = \sum_{t \in h, k'_t \geq -3} (\hat{W}_t[c_t] \ll (k'_t + 6)) + (\hat{W}_t[c'_t] \ll (k'_t + 6 - q_t)), \quad \text{out} = M' [\hat{y}_1; \dots; \hat{y}_H] + \text{bias},$$

the second term left out when $q_t > 3$, M' being M with $2^{e_{h,i}-6}$ folded into its columns; $|\hat{y}_h| \leq 2T \cdot 128 \cdot 2^{10} = 2^{25}$ fits 32-bit integers. The table read itself has no multiply; $\log_2 s_t$, q_t and c_{q_t} are a fixed handful of floating-point operations per table, independent of d . The tables of the six layers take 151.0 MB as 32-bit floats and 37.75 MB as int8. On an RTX 5090 the per-table integers come from a custom CUDA operator shared by training and inference, so training, the integer read and the exported model take identical integers; elsewhere a PyTorch definition computes the same integers. **Cost:** Table B row “3.2 int8” (derivation in note d): at eval 37,056 FLOPs and no MACs, against 3.2’s 135,168 FLOPs and 49,152 MACs; a training step 393,728 FLOPs and 147,456 MACs, against 3.2’s 384,512 and 147,456.

Result at 16K: quantised against unquantised

Same configuration, seed, data and trainer (3.2 +TV; code dd8c271e); only `quant_mode` differs: `exp_g_abl_46` (unquantised) final validation bpb **1.155175**, and `exp_g_abl_48` (p2_int8, trained on the CUDA operator, 1.09 h) **1.158890** (best = final for both). **The two curves run in parallel.** The gap opens by step 3,000 and then stays nearly constant, between +0.0024 and +0.0049 bpb over the remaining 13,000 steps, with the quantised variant above at all 32 evaluations (right panel). A least-squares fit of the gap from step 3,500 gives only a slight widening, +0.00005 bpb per 1,000 steps ($\approx +0.0007$ over the run). Quantisation therefore costs a small, nearly fixed offset, not a compounding degradation. **Its size at 16K is +0.0037 bpb**, above this cell’s seed spread (0.0015: `abl_05` 1.154739, `abl_42` 1.156272), so a real, reproducible cost rather than noise: the price of storing the tables in 37.75 MB instead of 151.0 MB. The earlier prototype of the same read agrees (`abl_45`, 1.159145).



Long-run results (48K steps)

The quantised variant (p2_int8) of 3.2 +TV at 48K steps, warmup and cosine schedule stretched to 48K, run on nebius (`exp_n_abl_47_B48k_light_lmfreezing_n2_tau0p5_tv10_tph128_seed1_p2int8`; training time 3.67 h): final validation bpb **1.123253** at step 48,000 and best **1.122593** at step 47,000, over 96 evaluations with the fixed evaluation (batch 48×100 steps, the first 12 rows skipped), the protocol of every other number in this document. The curve falls from 1.439334 at step 4,000 to 1.185099 at 16,000 and 1.125973 at 40,000, and is flat from step 43,000: its last eleven evaluations lie within 0.0009 (1.122593 to 1.123513), with no divergence. **Against the vanilla dense FFN at 48K** (untied, same fixed evaluation; `exp_n_0177_vanilla48k_corrected`, final 1.115420, best 1.115092 at step 47,500) the quantised variant is +0.007501 bpb above best against best and +0.007833 final against final.